



量子コンピューティング I-B

#10 FTQC編第二回

論理量子演算：スタビライザ・Steane 符号・論理ゲート

情報工学科 佐藤貴彦

今日の目標: 論理Qubitを動かす

2024 年、Microsoft社とQuantinuum社が 論理もつれ対（ベルペア）を、物理のもつれ対より低い誤り率で作った

- 符号化した方が誤り率が低い： いわゆる Break-even の達成
- この実験では継続的なエラー訂正ではなく、Post-selectionという”後追いによる成功確認”に留まっている

物理ベルペア
裸の 2 量子ビット

誤り率：高い

論理ベルペア
Steane 符号 x 1 4 物理量子ビット
= 2 論理量子ビット

誤り率：低い

Demonstration of logical qubits and repeated error correction with better-than-physical error rates

A. Paetznick,¹ M. P. da Silva,¹ C. Ryan-Anderson,² J. M. Bello-Rivas,¹ J. P. Campora III,² A. Chernoguzov,² J. M. Dreiling,² C. Foltz,² F. Frachon,¹ J. P. Gaebler,² T. M. Gatterman,² L. Grans-Samuelsson,¹ D. Gresh,² D. Hayes,² N. Hewitt,² C. Holliman,² C. V. Horst,² J. Johansen,² D. Lucchetti,² Y. Matsuoka,² M. Mills,² S. A. Moses,² B. Neyenhuis,² A. Paz,¹ J. Pino,² P. Siegfried,² A. Sundaram,¹ D. Tom,¹ S. J. Wernli,¹ M. Zanner,¹ R. P. Stutz,² and K. M. Svore¹

¹Microsoft Azure Quantum
²Quantinuum
(Dated: November 19, 2024)

The promise of quantum computers hinges on the ability to scale to large system sizes, e.g., to run quantum computations consisting of more than 100 million operations fault-tolerantly. This in turn requires suppressing errors to levels inversely proportional to the size of the computation. As a step towards this ambitious goal, we present experiments on a trapped-ion QCCD processor where, through the use of fault-tolerant encoding and error correction, we are able to suppress logical error rates to levels below the physical error rates. In particular, we entangled logical qubits encoded in the $[[7, 1, 3]]$ code with error rates $9.8\times$ to $500\times$ lower than at the physical level, and entangled logical qubits encoded in a $[[12, 2, 4]]$ code based on Knill's C4/C6 scheme with error rates $4.7\times$ to $800\times$ lower than at the physical level, depending on the judicious use of post-selection. Moreover, we demonstrate repeated error correction with the $[[12, 2, 4]]$ code, with logical error rates below physical circuit baselines corresponding to repeated CNOTs, and show evidence that the error rate per error correction cycle, which consists of over 100 physical CNOTs, approaches the error rate of two physical CNOTs. These results signify a transition from noisy intermediate scale quantum computing to reliable quantum computing, and demonstrate advanced capabilities toward large-scale fault-tolerant quantum computing.

<https://arxiv.org/abs/2404.02280>

今日はSteane符号・論理ゲートを組み合わせ、論理ベル状態を生成する

符号化の効用がノイズを上回る = break-even

符号化により、量子ビットも操作も所要時間も増える ▶ エラー要因は増大する
符号化のオーバーヘッドを実機上のエラー訂正で（特定の操作について）凌駕し始めている

- これは #9 で測った損益分岐（裸の誤り率 p と、符号化した $A \cdot p^2$ ）の実機版
- 2020年代に入りさまざまなbreak-evenが報告されるようになった
 - 量子ビットの長寿命化
 - 特定のゲートのエラー抑制 など
- 符号は星の数ほどあるが、今日は比較的シンプルな Steane 符号 を取り扱います
 - （最前線での流行は、表面符号からqLDPC符号に移行しつつある）

Sec.I

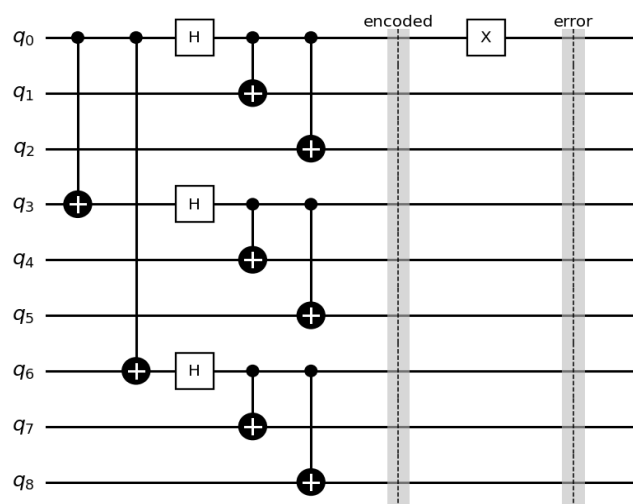
誤り訂正の復習

古典符号を視点に #9 を振り返る

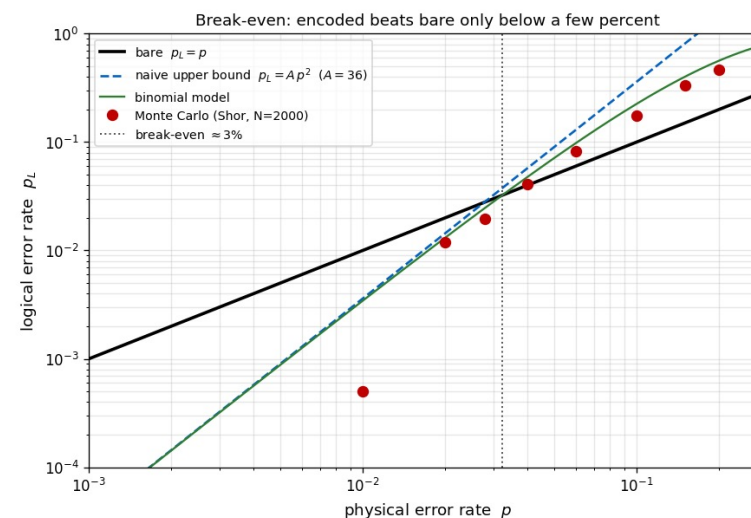
前回 #9 の到達点

#9 では 9 量子ビットの Shor 符号で、補助量子ビットを使って $Z_1 Z_2$ などを読み、固有値 ± 1 （隣が同じか・違うか）を読んで誤りを直した

■ 誤りは直せた。符号化 → エラー蓄積 → 測定 → 訂正、までは前回到達済み



符号化回路とXエラー

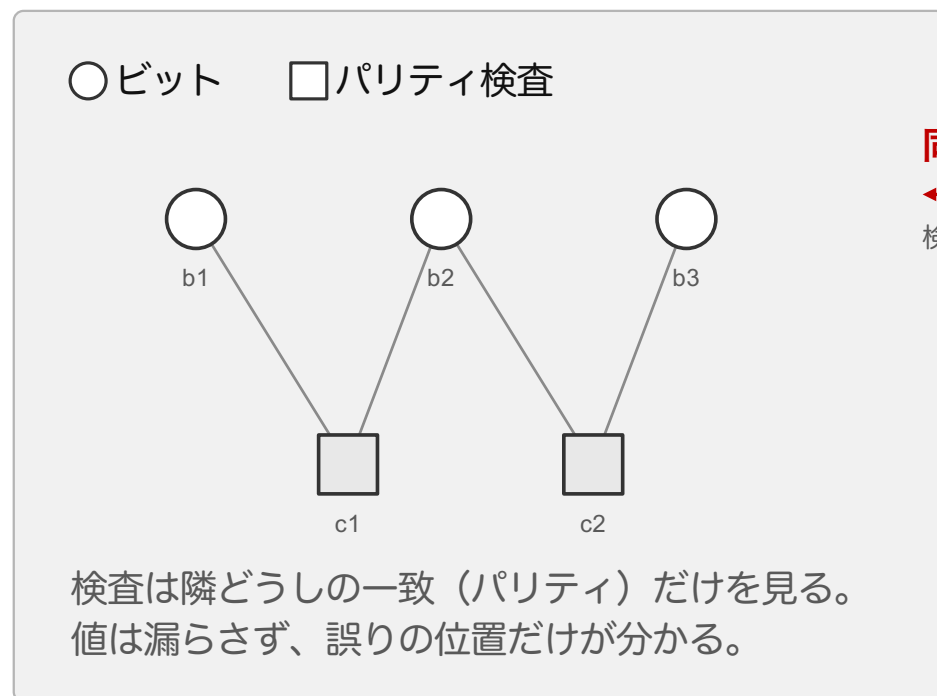


break-even point (の上界)を調べた

何を測っていたのか

シンドローム測定によって量子状態そのものには触れず、誤りの位置だけが分かった

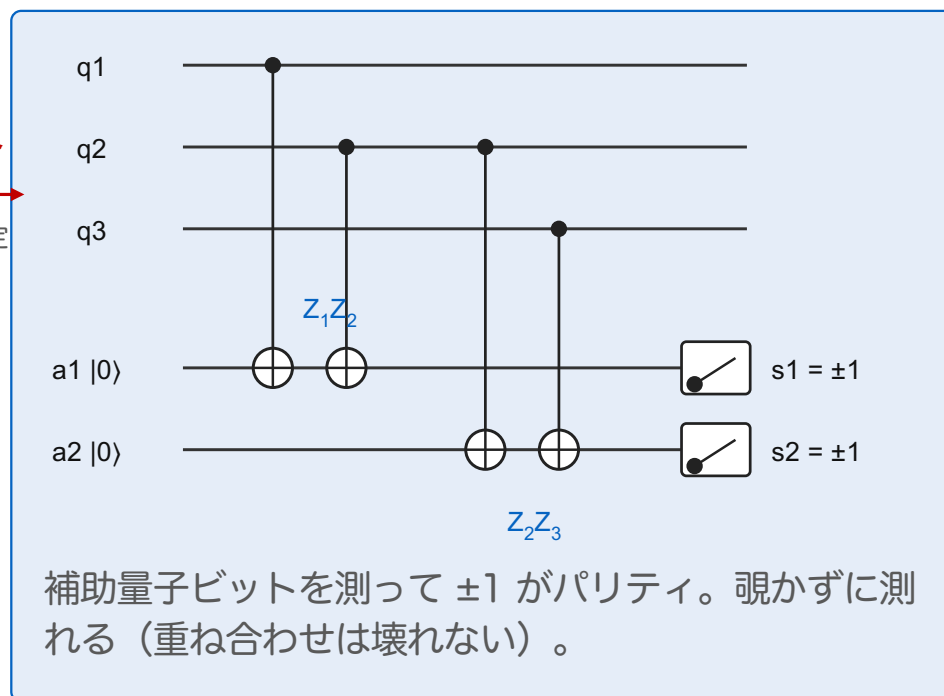
古典：3 ビット繰り返し符号 (Tanner グラフ)



同じ骨格

検査 ↔ 測定

#9：シンドローム抽出回路（量子）

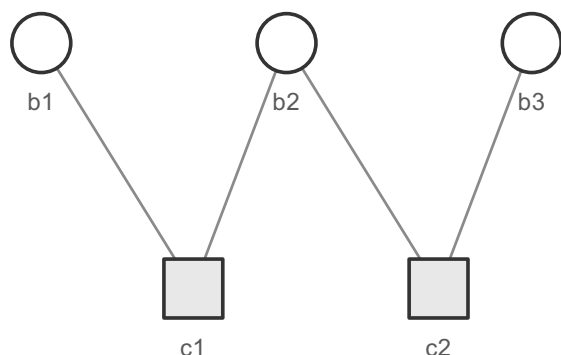


古典の繰り返し符号とパリティ検査

0 を 000、1 を 111 と 3ビットで符号化する

- 誤りを探すには「隣どうしが一致しているか」だけを尋ねればよい：パリティ検査
- パリティ検査を用いると、元が 000 か 111 かは分からないまま、誤りの位置だけ分かる。

○ビット □パリティ検査



検査行列 H (2×3) と生成行列 G (1×3) :

$$H_{\text{rep}} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \quad G_{\text{rep}} = [1 \ 1 \ 1]$$

G で符号化されたビット列 c は $Hc^T = 0$ を満たす
 000・111 はシンδροーム 0、単一ビット反転は
 位置ごとに異なるシンδροーム ($[1,0]$ / $[1,1]$ /
 $[0,1]$) を返すので 1 ビット誤りを特定できる

#9 のシンドローム抽出 = 古典の骨格 + 覗かずに測れる

パリティ検査が返すのは一致・不一致のパターン（シンドローム）だけ

▶ 位置は教えるが中身は漏らさない。

#9 のシンドローム抽出は、この古典の骨格に 「測っても重ね合わせが壊れない」 を足したもの

H_X (X型, 2×9) 位相反転を検出

```
1 1 1 1 1 1 0 0 0
0 0 0 1 1 1 1 1 1
```

H_Z (Z型, 6×9) ビット反転を検出

```
1 1 0 0 0 0 0 0 0
0 1 1 0 0 0 0 0 0
0 0 0 1 1 0 0 0 0
0 0 0 0 1 1 0 0 0
0 0 0 0 0 0 1 1 0
0 0 0 0 0 0 0 1 1
```

量子ビットへのエラーは
XエラーとZエラーに分解できるので、
shor符号はX型・Z型の検査行列を
シンドローム測定で実装している

古典側のツール（検査行列・シンドローム復号）は秋学期「符号理論」で本格的に扱う（はず）
今日は量子ゲートと相性が良い、新たな量子誤り訂正符号を導入する

Sec.II

スタビライザの導入

符号や量子状態を整理するツールがある

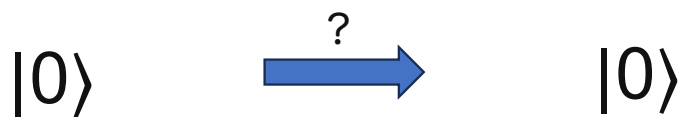
量子状態を、少数の演算子で書き下したい

n 量子ビットの状態は振幅が 2^n 個.

#9 で見たように、誤り訂正は多くの物理量子ビットを用いる

(ケットベクトルを用いて) 振幅の列で扱うと、手に負えない状態になってしまう

- 振幅の列の代わりに、その状態に作用させても何も起きない = スタビライズ (固定) する演算子を導入する. この演算子をスタビライザと呼ぶ (群論の専門用語ですね).
- 「どんな状態か」を、「何がそれをスタビライズするか」で言い換える
まず一番単純な、 $|0\rangle$ を変化させない演算 を考えよう



状態を変化させない演算子は何だろうか？

“演算子”は量子状態の変換操作全般を指す
量子ゲートは“ユニタリな”演算子を指す

Z が $|0\rangle$ を、X が $|+\rangle$ をスタビライズする

Z は $|0\rangle$ を +1 倍で素通りさせてスタビライズする

▶ $Z|0\rangle = +|0\rangle$ 、 $Z|1\rangle = -|1\rangle$ なので固有値は ± 1
同様に、X は $|+\rangle$ をスタビライズする。

$|1\rangle$ は $-Z$ でスタビライズされている

■ #9 で測った固有値 ± 1 は、まさにこの「スタビライズ」の符号だった

■ $+1$ = スタビライズされている / -1 = スタビライズされていない

■ 状態が変われば、スタビライズする演算子も変わる。

■ では、もつれ状態は何がスタビライズするのか？

片方を触ると状態が変わりそうだが・・・

$$\frac{1}{\sqrt{2}} (|00\rangle + |11\rangle) \xrightarrow{?} \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

同じ 2 演算子の符号で、4 つのベル状態を名指す

ベル状態をスタビライズするのは $X \otimes X$ と $Z \otimes Z$ の二つ(どちらも非局所操作)。

✓ この同じ 2 演算子の固有値 ± 1 の組み合わせで、4 つのベル状態が区別できる。

ベル状態	式	XX	ZZ	YY	安定化生成子
$ \Phi^+\rangle$	$\frac{ 00\rangle + 11\rangle}{\sqrt{2}}$	+1	+1	-1	$\langle +XX, +ZZ \rangle$
$ \Phi^-\rangle$	$\frac{ 00\rangle - 11\rangle}{\sqrt{2}}$	-1	+1	+1	$\langle -XX, +ZZ \rangle$
$ \Psi^+\rangle$	$\frac{ 01\rangle + 10\rangle}{\sqrt{2}}$	+1	-1	+1	$\langle +XX, -ZZ \rangle$
$ \Psi^-\rangle$	$\frac{ 01\rangle - 10\rangle}{\sqrt{2}}$	-1	-1	-1	$\langle -XX, -ZZ \rangle$

n Qubitの状態を振幅で書けば 2^n 個の項を要するが、スタビライザなら n 本で書き下せる

✓ スタビライザ同士の積は他のスタビライザになるが、全てのスタビライザを作れる最小集合をジェネレータ(生成子)と呼ぶ：ここではXXとZZ

XX と ZZ の固有値 (±) の 4 通りが 4 状態に一対一対応 / YY = -XX · ZZ (従属量)

■ 固有値 +1 ■ 固有値 -1

■ ジェネレータが n 本 ある ▶ 量子状態は一意に定まる

■ ジェネレータが k 本 ($k < n$) ある ▶ 量子状態は $n-k$ 量子ビット の自由度を持つ

NISQ編までは $k=0$ だっ

ジェネレータ（生成子）でshor符号を定義する

#9 で補助量子ビットを使って測っていた $Z_1 Z_2$ などは Shor 符号を定義する 8 本のジェネレータ だった

✓ ジェネレータから生成されるスタビライザ群は符号空間を+1に固定する

✓ shor符号の自由度 = ジェネレータの+1同時固有空間の次元は $2^2 = 4$ 量子ビットの情報が書き込める

生成子	種類	パウリ表記	作用ビット	役割
S1	Z 型	$Z_0 Z_1$	q_0, q_1	ブロック A 内のビット反転検出
S2	Z 型	$Z_1 Z_2$	q_1, q_2	ブロック A 内のビット反転検出
S3	Z 型	$Z_3 Z_4$	q_3, q_4	ブロック B 内のビット反転検出
S4	Z 型	$Z_4 Z_5$	q_4, q_5	ブロック B 内のビット反転検出
S5	Z 型	$Z_6 Z_7$	q_6, q_7	ブロック C 内のビット反転検出
S6	Z 型	$Z_7 Z_8$	q_7, q_8	ブロック C 内のビット反転検出
S7	X 型	$X_0 X_1 X_2 X_3 X_4 X_5$	$q_0 \sim q_5$	ブロック A-B 間の位相反転検出
S8	X 型	$X_3 X_4 X_5 X_6 X_7 X_8$	$q_3 \sim q_8$	ブロック B-C 間の位相反転検出

情報量が多くなってきたので、
次ページで整理してみましょう

まとめ：ジェネレータ（独立したスタビライザ）の本数と、残る自由度

1 量子ビット
状態 $|0\rangle \cdot |+\rangle$

$|0\rangle$ を Z が、 $|+\rangle$ を X がスタビライズする
 $Z|0\rangle = +|0\rangle$ (固有値 +1)

スタビライザ 1 本
自由度 0 (1 状態に固定)

2 量子ビット
ベル状態 $|\Phi^+\rangle$ ほか

$X \otimes X$, $Z \otimes Z$ がスタビライズする
同じ 2 演算子の符号 ± 1 で 4 状態を示せる (自由度 0)
▶ スタビライザが 1 本なら 1 Qubit 分の自由度がある

状態	XX	ZZ
$ \Phi^+\rangle$	+1	+1
$ \Phi^-\rangle$	-1	+1
$ \Psi^+\rangle$	+1	-1
$ \Psi^-\rangle$	-1	-1

9 量子ビット Shor 符号
#9 で使った符号

$$|0_L\rangle = (|000\rangle + |111\rangle)^{\otimes 3} / 2\sqrt{2}$$

振幅で書けば 2^9 個 $\leftrightarrow$ ジェネレータは 8 本

ジェネレータ 8 本（誤りを調べるために「測る」相手） ■ X 型 ■ Z 型

Z 型 6 本 (X 誤りを検出)



X 型 2 本 (Z 誤りを検出)



この 8 本は、#9 で補助量子ビットを使って測っていた演算子 (Z_1Z_2 等)。符号を定義するジェネレータそのものだった。

ジェネレータを減らすと、決まり切らない 空間 が残る

ジェネレータがQubit数と同数になると、状態は決定される＝自由度がない

1 本減らすと、決まり切らない 2 次元の空間 が残る = これが論理量子ビット 1 個分の自由度

- 符号化された空間の構造を決めるのがジェネレータ（独立なスタビライザ）
- 符号化された状態を動かすのが 論理演算子(論理Zゲート = Z_L など. 後述)
- ジェネレータが多いほど部屋は狭く、少ないほど広い(載る論理量子ビットが多い)
- 以下の特徴を合わせ持つと”良い符号”だが、トレードオフも大きく全てを満たすことは難しい
 - ✓ 少ないジェネレータで、高いエラー耐性を持つ
 - ✓ 論理演算子が実行しやすい
 - ✓ スタビライザ測定が実行しやすい

スタビライザは論理量子ビットを触れない

スタビライザは切り分けた空間の内部（論理量子ビットの状態）を操作できない ので、測ると「どの部分空間にいるのか(誤りの有無)」は分かるが、「どんな論理状態なのか」は分からない

■ だからシンドローム抽出はデータを壊さない。誤りの有無だけを抽出し、中身は守られている

■ なぜ触れないかの厳密な理由(可換性)は副読本 第II部(予定)へ。

■ 第I部（Shor符号）はK-LMSにリリースされています。

測定の準備 — 固有空間・基底変換・可換性	3
道具 1 — 測定とは、固有空間への選り分けである	3
道具 2 — 向きを変えれば、測定は計算基底ひとつで足りる	5
道具 3 — 二体の検査: ビットのパリティと位相のパリティ	6
道具に慣れる — 小演習	8
第 1 章 シンドローム測定は射影測定である	11
① 問い — 講義が一段で跳んだ場所	11
② 観察 — Z_1Z_2 という行列は、空間をどう割るか	12
③ 一般化 — 一つの幾何が、状態を守り、誤りを離散化する	14
④ 演習 — 導出を疑って、検算する	18
結び — 一つの幾何、二つの帰結、そして次の問い	18
第 2 章 基底の双対 — $\pm$ の比較はなぜ位相を見るのか	19
① 問い — 構造は持ち帰った、機構が空白	19
② 観察 — H を双対性として読み、ブロックの $\pm$ を Z の $0/1$ にする	20
③ 一般化 — 同じ機構が、双対な基底でもう一度働く	21
④ 演習 — 双対を、符号まるごとで検算する	23
結び — 一つの基底変換、双対に開く二つの符号、そして残った糸	24

第 3 章 縮退 — 区別できない誤りを、それでも直せる理由	25
① 問い — 講義が名を伏せて通った場所	25
② 観察 — 三つの位相反転は、符号語に同じ作用をする	26
③ 一般化 — 符号空間を動かさない違いは、訂正の上で消える	27
④ 演習 — 区別できなさを、手で確かめる	29
結び — 区別しない余裕、そして次の問い	29
第 4 章 訂正装置も誤る — フォールトレランスの設計	31
① 問い — 完璧な訂正装置という前提を外す	31
② 観察 — 誤りは $CNOT$ をどう伝うか	32
③ 一般化 — 増殖を遮断する回路の組み方	34
④ 演習 — 壊して、踏みとどまるところを見る	36
結び — 増殖は止めた、では勝てる誤り率はどこか	37
第 5 章 しきい値定理 — 「符号を大きくするほど下がる」境界の由来	39
① 問い — 符号の上に、また符号を積む	39
② 観察 — 一段ごとに、二乗が積み重なる	40
③ 一般化 — 閉じた式、境界、そして二つの別物	41
④ 演習 — 算術を疑って、検算する	44
結び — 第 I 部を閉じる	46

Sec.IIのまとめ

2 量子ビット

スタビライザは **1 本だけ**

$$X \otimes X = +1$$

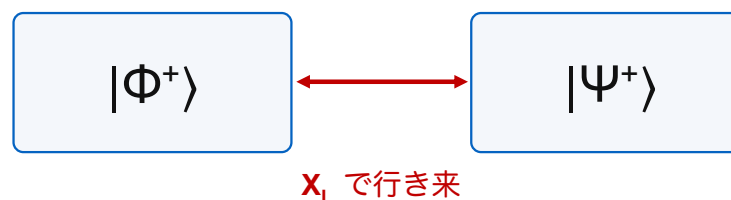
$$2 - 1 = 1$$

残った自由 1
= 2 次元の部屋

= 論理量子ビット 1 個

壁 = スタビライザ $X \otimes X = +1$

符号空間（部屋） = 2 次元



2 状態の「向き」を見分けるのが $Z_L (= Z \otimes Z)$ 。壁の中の自由が論理量子ビット。

もう 1 本足すと

$Z \otimes Z = +1$ も課すと

部屋は 1 点に縮む

→ $|\Phi^+\rangle$ に固定

(= 前ページのベル状態)

スタビライザが多いほど部屋は狭い。少ないほど広く、載る論理量子ビットが多い。

符号の構造を整理する：[[n, k, d]] 記法

n 個の物理量子ビットに s 本のスタビライザをかけると、残る論理量子ビットは $k = n - s$ 個
このとき、符号の構造は $[[n, k, d]]$ で表す(d = 符号距離: 半分を超える誤りは直せない)

■ #9で学んだ Shor = $[[9, 1, 3]]$ ($9-8=1$)

■ #10で学ぶ Steane = $[[7, 1, 3]]$ ($7-6=1$)。

✓ どちらも論理 1 個・距離 3。

■ 同じ論理 1 個・距離 3を実現するのに何が違うのだろうか？

■ 物理量子ビットが 9 は 7 より冗長に思われる

■ また、より量子ビットが少ない $[[5, 1, 3]]$ 符号も #9 で名前だけ見たが、より効率的なのだろうか？

次セクションは $[[7,1,3]]$ の Steane符号を見てみよう

Sec.III

計算に向けた符号

古典 $[7,4,3]$ Hamming から、計算に向けた Steane へ

古典 [7,4,3] Hamming 符号

距離 3 の符号を作るだけなら 3 ビット繰り返し符号の多数決で直せる
では なぜ 7 ビットも使うのか [7,4,3] Hamming 符号が実用化されているのだろうか？

- 答えは効率で、古典 [7,4,3] Hamming は
 - ✓ 3 つの検査だけで 7 ビット中の 1 つの誤りを位置まで特定し、
 - ✓ 4 ビットの情報を書き込むことができる

- ここで $[n,k,d]$ は

- ✓ $[]$ 線形符号
- ✓ n 符号長のビット数
- ✓ k データのビット数
- ✓ d 最小ハミング距離 を表す

これから見るように、
量子の符号は $[[n, k, d]]$
と二重カッコを用いる

参考 | 古典 [7,4,3] Hamming 符号の生成行列Gと検査行列H

- 生成行列を用いてビット列を符号化し、
- 検査行列を用いて誤り箇所を特定することができる

	q0	q1	q2	q3	q4	q5	q6
g1	1	1	1	0	0	0	0
g2	1	0	0	1	1	0	0
g3	0	1	0	1	0	1	0
g4	1	1	0	1	0	0	1

生成行列 G
4×7・符号化
(4 情報 → 7 符号)

h1	0	0	0	1	1	1	1
h2	0	1	1	0	0	1	1
h3	1	0	1	0	1	0	1

検査行列 H
3×7・誤り検出

$$GH^T = 0$$

古典 [7,4,3] Hamming : 検査だけで 7 ビット中の誤り 1 つを位置まで絞る

3 つの検査 (各検査は決まった 4 ビットを見張る)

	1	2	3	4	5	6	7
検査1	○	○	○	●	●	●	●
検査2	○	●	●	○	○	●	●
検査3	●	○	●	○	●	○	●

● = この検査が見張るビット

3 ビット繰り返し符号は 1 ビットを 3 個使って 1 つ守る。
Hamming は 7 ビットで 4 つ守れる (直せる強さは同じ単一誤り)

検査の結果 (シンドローム) → 誤り位置の対応表

(検査1 検査2 検査3 を 2 進数として読むと位置)

シンドローム	誤り位置
0 0 0	誤りなし
0 0 1	1
0 1 0	2
0 1 1	3
1 0 0	4
1 0 1	5
1 1 0	6
1 1 1	7

この対応表は演習①の古典デコーダ (シンドローム → 位置) そのもの。量子のSteane符号もこの仕組みを使っている

[7,4,3]Hamming符号を元に、[[7,1,3]]Steane符号が作られた

[[9,1,3]] Shor 符号の特徴

- 史上初の量子訂正符号
 - X と Z のスタビライザが非対称
 - ✓ 位相反転の検査が重み6と大きい
 - ✓ Hゲートがナイーブに実装できない
- など、運用上の問題も抱えていた

[7,4,3] Hamming を X エラー用・Zエラー 用に並列運用し、
より量子計算に適したSteane符号 が提案された



Andrew Steane

英国

1965~

Oxford・Clarendon Laboratory (物理)
量子誤り訂正で Maxwell Medal (IoP, 2000)

量子の誤りに対応するため、スタビライザも二系統用意する

古典の誤りは 1 種類(ビット反転)。量子は $X \cdot Z$ の 2 種類。だからスタビライザも X 型・ Z 型の二系統が要る。

- 量子符号は古典符号を二枚貼り合わせたもの、と言える。
- では Steane はどの二枚を貼ったのか。次のセクションで開ける。
- 検査の結果(シンδροーム)を 2 進数として読むと、そのまま誤り位置になる。これが演習①の古典デコーダ。
- 量子は X と Z の二系統を見張るので検査が倍($3 \rightarrow 6$)。同じ 7 ビットで守れるのは $7 - 6 = 1$ 量子ビット。場面2 の数え方に直結する。
- 検査行列とシンδροーム復号を本格的に扱うのは秋学期「符号理論」。

Steane符号は 6本のジェネレータからなる（自由度は1 量子ビット）

古典の誤りは1種類だが量子は $X \cdot Z$ の 2 種類なので、スタビライザも X 型・ Z 型の二系統が必要

各ジェネレータが見張る量子ビット（番号付け規約を固定 = 全資料で統一）

X 型 3 本 (X 誤りを見張る)

	1	2	3	4	5	6	7
g1	○	○	○	●	●	●	●
g2	○	●	●	○	○	●	●
g3	●	○	●	○	●	○	●

Z 型 3 本 (Z 誤りを見張る)

	1	2	3	4	5	6	7
g4	○	○	○	●	●	●	●
g5	○	●	●	○	○	●	●
g6	●	○	●	○	●	○	●

復習：ジェネレータはそれぞれ独立（積で作れない）

6 本が互いに独立 $\rightarrow 7 - 6 = 1$ 量子ビット分の自由度が残る = 論理量子ビットとして使える

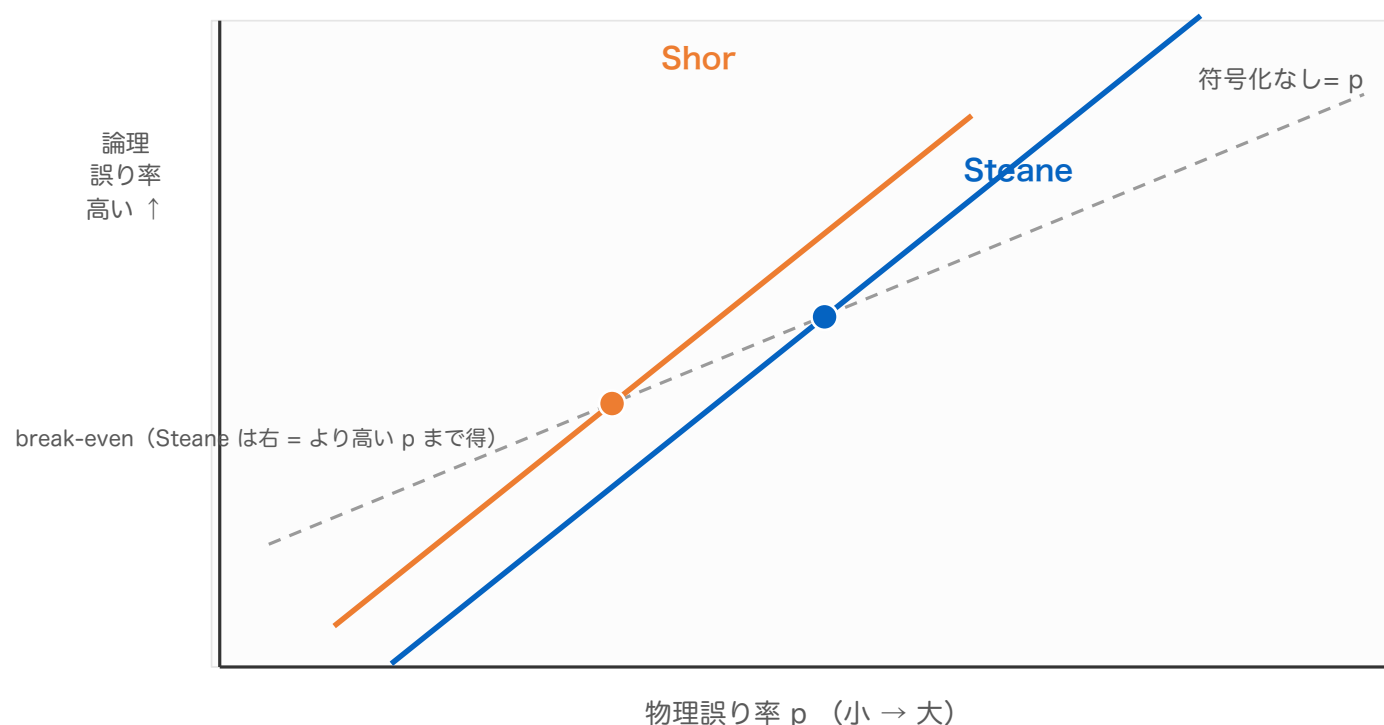
ジェネレータはそれぞれ可換（同時に測れる）

どの 2 本も重なりが偶数 $\rightarrow$ 同時に測れる
 $\rightarrow$ 共通の $+1$ 固有空間 = 符号空間。
 #9 で $Z_1 Z_2$ を一度に測れたのと同じ筋。

検査は古典[7,4,3] Hamming符号の倍
 書き込める情報量は1/4の1量子ビット・・・
 （最近は[[144,12,12]]などもある [Link](#)）

ShorとSteaneの性能を比較する

距離は同じだが、Qubit数が違うため、係数 = 曲線の高さが異なる



ノートブックで実行し、この概形通りの結果になるか確かめ、考察しよう

両方とも距離 3

→ 傾きは同じオーダーになる

Steane は係数が小さく曲線が下
= 同じ強さでも、量子ビットが少なく済み、より低い誤り率になるはず。

パラメータ依存な部分もあり、
閾値はゆるい上界となっている

代表的な距離 3 の符号

符号	$[[n,k,d]]$	CSS	Clifford が transversal	特徴
Shor (1995)	$[[9,1,3]]$	○	△ (X/Z 非対称で不便)	史上初・9 量子ビットで冗長性が大きい Link
5 量子ビット (1996)	$[[5,1,3]]$	× (非 CSS)	× (一式は揃わない)	Laflammeら、Bennetらが独立に発見 Link1 , Link2 最小・完全符号 (冗長性なし)
Steane (1996)	$[[7,1,3]]$	○ (自己双対)	○ (H・S・CNOT 全部)	今日の主題 X と Z を独立に検出できる・計算向き

距離はどれも 3だが、
[[5,1,3]]はどれか一つのQubitに発生したXorYorZエラーしか検知できない (16分類しか出来ないため)
Steane は X と Z を独立に検出でき (64分類)、Clifford が全部 transversal (Sec.III) なので、論理計算を学ぶのに向いている

Sec.IV

トランスバーサルな論理ゲート

論理量子ビットの上で計算する

論理量子ビットを操作したい

論理量子ビットの状態を変化させる量子ゲートはどう実装すればよいだろうか？

例：steane符号化された 論理 $|0\rangle$ と 論理 $|1\rangle$ を入れ替える = 論理Xゲート はどうやって実現する？

$$|0_L\rangle = (1/\sqrt{8}) (|0000000\rangle + |0001111\rangle + |0110011\rangle + |0111100\rangle \\ + |1010101\rangle + |1011010\rangle + |1100110\rangle + |1101001\rangle)$$



論理X = X_L は「7 個全部に同じXゲート」で実現できる
たしかめてみよう

$$|1_L\rangle = (1/\sqrt{8}) (|1111111\rangle + |1110000\rangle + |1001100\rangle + |1000011\rangle \\ + |0101010\rangle + |0100101\rangle + |0011001\rangle + |0010110\rangle)$$

- 「全ての量子ビットに同じゲート」という一つの操作で実現する論理ゲートを Transversalなゲートという
- ✓ 局所的なゲートの組み合わせで論理ゲート実装できると、運用上の利点大きい（後述）

トランスバーサルな論理ゲート

$X_L \cdot Z_L \cdot H_L$ はSteane符号におけるトランスバーサルなゲートとして知られている（他にもある）

$$|0_L\rangle = (1/\sqrt{8}) (|0000000\rangle + |0001111\rangle + |0110011\rangle + |0111100\rangle \\ + |1010101\rangle + |1011010\rangle + |1100110\rangle + |1101001\rangle)$$

$$|1_L\rangle = (1/\sqrt{8}) (|1111111\rangle + |1110000\rangle + |1001100\rangle + |1000011\rangle \\ + |0101010\rangle + |0100101\rangle + |0011001\rangle + |0010110\rangle)$$

論理 $|0\rangle$ は偶数個の1を持ち、論理 $|1\rangle$ は奇数個の1を持つので、
論理 $Z = Z_L$ は X と同様にトランスバーサルに実行できる

これは偶然ではなく、Steane が自己双対(X と Z を入れ替えても自分に戻る) なため
同様に、 $X \leftrightarrow Z$ を入れ替える論理 H もトランスバーサルに実装できる▶

$$|+_L\rangle = (1/4) \sum_{\{c \in \text{Hamming}[7,4,3]\}} |c\rangle$$

$$= (1/4) (|0000000\rangle \\ + |0010110\rangle + |0011001\rangle + |0100101\rangle + |0101010\rangle + |1000011\rangle + \\ |1001100\rangle + |1110000\rangle \leftarrow \text{重み3} \\ + |0001111\rangle + |0110011\rangle + |0111100\rangle + |1010101\rangle + |1011010\rangle + \\ |1100110\rangle + |1101001\rangle \leftarrow \text{重み4} \\ + |1111111\rangle)$$

【実験6】

トランスバーサルゲートの利点と限界

横一列が論理ゲートになる理由と、誤りに強い理由は、符号設計に由来している

- (i) 論理ゲートになる：ビット一つずつの単純操作が、ブロック全体として論理演算子を正しく動かす
- (ii) 誤りが広がらない：符号空間内にエラーが波及しないので、各ブロックは 1 誤りのまま(訂正できる)

■ 利点

- 実装が単純：アンシラやゲートテレポーテーションなどが不要
- リソースオーバーヘッドが少ない：魔法状態蒸留（#11）などの複雑な操作がいらぬ

■ 限界

- Eastin-Knillの定理：普遍的なゲート集合を全てトランスバーサルに実装することは不可能

論理状態の読み出し

Steane符号の論理量子ビットはどうやって読み出すのだろうか

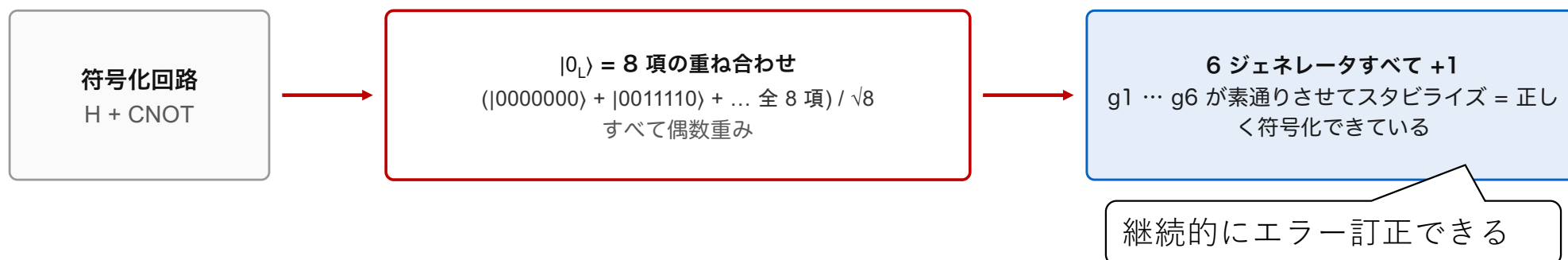
$$|0_L\rangle = (1/\sqrt{8}) (|0000000\rangle + |0001111\rangle + |0110011\rangle + |0111100\rangle \\ + |1010101\rangle + |1011010\rangle + |1100110\rangle + |1101001\rangle)$$

$$|1_L\rangle = (1/\sqrt{8}) (|1111111\rangle + |1110000\rangle + |1001100\rangle + |1000011\rangle \\ + |0101010\rangle + |0100101\rangle + |0011001\rangle + |0010110\rangle)$$

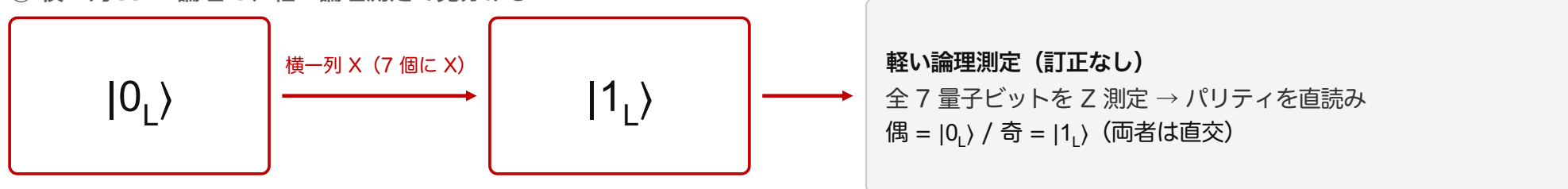
- 基本的には全ての量子ビットを測定し、出力を見ればよい
 - ✓ 1010101であれば論理0だったはず
 - ✓ 0101010であれば論理1だったはず
 - ✓ (論理0・論理1に含まれないビット列はハミング距離が近いものの可能性が高い)

トランスバーサルな論理量子計算の流れ

① 符号化 → 論理 0



② 横一列 X → 論理 1、軽い論理測定で見分ける



論理 0 と論理 1 を、訂正なしのパリティ読みで見分けられる (符号空間も量子状態も壊れてしまう)
 アンシラ量子ビットを使った間接測定で符号空間を壊さない 非破壊読み出し をすることもできる (副読本・予定)

演習 ① | 誤りを撒いて、古典デコードで直す

所要 8 分 | ノートブック: lec10_exercise.ipynb | 書くのは 数行

python(擬似)

```
# |0_L> を組み、Z 型 6 ジェネレータを測って Z 型シンδροームを得る(提示済)
z_syndrome = measure_Z_stabilizers(state) # 例: [b4, b2, b1]

# TODO: Z 型シンδροーム -> X 誤りの位置 の対応を埋める
error_pos = _____ # syndrome から位置を決める
corrected = apply_X(state, error_pos) # 訂正
assert fidelity(corrected, ideal) == 1.0 # 復元を確認
```

ヒント Z 型シンδροームは「どの Z 型検査が外れたか」の結果に対応している
2 進で読むと X 誤りの位置番号になる（一意に決まる）ので訂正してみよう

演習 ① 解答 | 量子の誤りを古典の表引きで直す

```
# Z 型シンδροーム [b4,b2,b1] を 2 進数として読むと X 誤りの位置
pos = (b1 << 0) | (b2 << 1) | (b4 << 2) # 0 = 誤りなし
if pos != 0:
    corrected = apply_X(state, pos - 1) # 位置 pos の X 誤りを打ち消す
# -> 訂正後、忠実度 = 1.0000(元の論理状態へ復元)
```

python

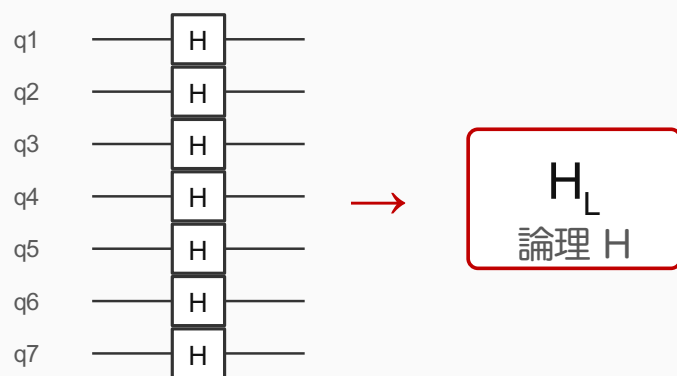
量子のシンδροームを、古典的に処理し、エラーを訂正した読み出しもまた、古典符号が仕事をしている

このような古典処理は「デコーダ」と呼ばれる

Steane符号のデコーダは Hamming符号に対応した表の確認だが、エラー訂正能力の高い符号はデコーダの負荷が大きくなる

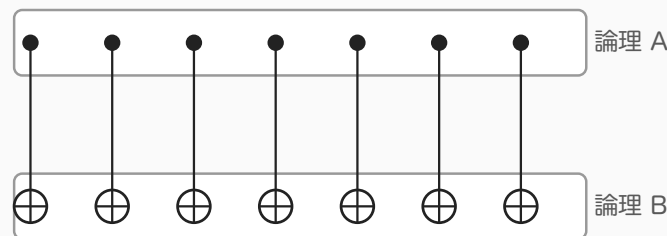
CNOTゲートもトランスバーサルゲートとして実行できる

1 量子ビットゲートの例：論理 H



物理 H を 7 個すべてに掛ける ($H \otimes 7$)

2 量子ビットゲートの例：論理 CNOT



i 番目どうしを CNOT (7 本) = CNOT_L (論理 CNOT)

CSS符号： 同じ構造の純X・純Zスタビライザからなる符号は一般的にトランバーサルCNOTゲートが実行可能

▶ Steane符号は代表的なCSS符号のひとつ

▶ 今日学んだトランスバーサルな論理ゲート (H、CNOT) を組み合わせて量子もつれを作ってみよう

演習 ② | transversal CNOT で論理ベル対を組む

所要 5 分 | ノートブック: lec10_exercise.ipynb | 書くのは 数行

python(擬似)

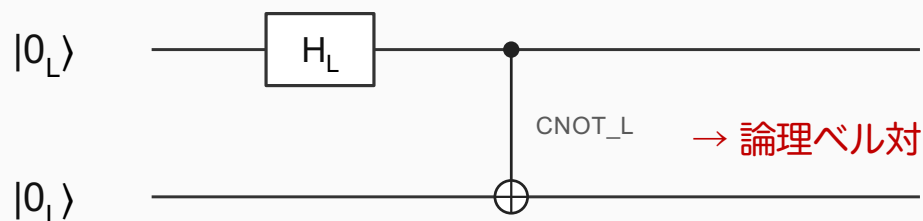
```
# 2 つの Steane ブロック A, B を用意し、A に論理 H をかけた(提示済)
apply_HL(state, block=A)

# TODO: ブロック間に論理 CNOT(= 7 個の物理 CNOT)をかける
for i in range(7):
    _____ # A[i] を制御、B[i] を標的に CNOT
# -> Z_L Z_L, X_L X_L を測ると両方 +1(完全相関)になるはず
```

ヒント 論理 CNOT は特別な回路ではない。
符号化されたブロック A の i 番目とブロック B の i 番目を、ビットごとに物理 CNOT でつなぐだけ。

発展 | 論理スタビライザで論理ベル対の基本相関を確かめる

論理回路



スタビライザを測れば、ベル対として振る舞うはず

論理スタビライザ (Pauli 相関)

$$Z_L \otimes Z_L = +1$$

$$X_L \otimes X_L = +1$$

どちらも +1 に確定 = 論理ベル対となっている。

概念1 で見たベルスタビライザ ($X \otimes X$, $Z \otimes Z$) が、論理レベルでそのまま成り立っている。

一致率 = 1.0000 ▶ 論理量子ビットでもどうやらもつれている (これだけでは断言出来ない. 検証は来週)

トランスバーサルなゲートでは万能量子計算が実現できない

Steane符号でトランスバーサルに作れるゲート = X, Y, Z, S, CNOT は全て Cliffordゲートに含まれる
耐性ある論理ゲートは揃ったが、基礎編で見たとおり Clifford だけでは万能ではない

✓ Gottesman-Knillの定理： Cliffordゲートのみの回路は古典で効率的にシミュレーションで可能

✓ H_L (アダマール)

transversal で作れる

✓ S_L (位相)

transversal で作れる

✓ CNOT_L

transversal で作れる

× T_L (45°回転・非 Clifford)

transversal では作れない

万能な論理量子計算を実現するには

符号がもつ対称性のおかげで、 $H \cdot S \cdot CNOT$ は横一列に掛けるだけで論理ゲートになったが…

- 非 Clifford ゲートを 1 つ(標準は Tゲート、Z軸周りの45 度回転)足す必要がある
- その T が、Steane では transversal にならず作りにくい。
 - ✓ Tをトランスバーサルに作れる符号（3次元カラー符号など）もあるが、エラー訂正が難しいなどのトレードオフがある
- #11では 魔法状態 とされる量子状態を用いてSteane符号上の万能量子計算を実現します
 - ✓ （中間レポートも出る予定です）

今日はここまで、おつかれさまです!

誤り訂正を一つの言語で書き直し、計算に向けた Steane の上で論理ゲートが transversal に動き出した。

1

スタビライザという一つの言語

誤りを測る = 符号を定義するジェネレータを測ること。古典パリティ検査と同じ骨格だった。

2

計算に向けた Steane を選ぶ

距離は Shor も 5 量子ビットも同じ 3。Steane は CSS・自己双対で、論理計算が素朴にできる。

3

論理ゲートは transversal に動く

物理ゲートを横一列に当てるだけ。素朴さがそのまま fault-tolerant。足りない一個(非 Clifford T)は #11。

次回 #11: 足りない一個のゲートを作る(magic state と非 Clifford、そして万能へ)